# Practical No.1(Vaccum Cleaner)

**Aim:-** Design a simple rational agent by defining PEAS for vacuum cleaner environment and autonomous taxi. Classify enviornment. Implement a simple table driven agent in python.

**Code:-**

```
class TableDrivenVacuumAgent:
    def __init__(self):

        self.percept_history=[]
        self.table={
            #Time Step 1:Single percept sequences.
            ((('A','Clean'),),):'Right',
            ((('A','Dirty'),),):'Suck',
            ((('B','Clean'),),):'Left',
            ((('B','Dirty'),),):'Suck',
            #Time Step 2:Two percept sequences(examples of history).
            ((('A','Clean'),),(('B','Dirty'),)):'Suck',
            ((('A','Clean'),),(('B','Clean'),)):'Right',
            ((('B','Dirty'),),(('A','Clean'),)):'Right',
            ((('B','Dirty'),),(('B','Clean'),)):'Left',

            #Time Step 3:Three percept sequences
            ((('A','Dirty'),),(('A','Clean'),),(('B','Dirty'),)):'Suck',
            ((('A','Dirty'),),(('A','Clean'),),(('B','Clean'),)):'Left',
            }

    def act(self,current_percept):
        self.percept_history.append(current_percept)
        #Convert history list to tuple of tuples so it can be used as dictionary key.
        history_key=tuple(self.percept_history)
        action=self.table.get(history_key,'Right')
        return action
```

```
if __name__=="__main__":
    agent=TableDrivenVacuumAgent()
    #Step 1:Agent starts in room A, which is dirty.
    percept_1=('A','Dirty')
    action_1=agent.act(percept_1)
    print(f"Percept:{percept_1}->Action:{action_1}")

    #Step 2:Agent sucked the dirt, so the room A is now cleaned.
    percept_2=('A','Clean')
    action_2=agent.act(percept_2)
    print(f"Perecpt:{percept_2}->Action:{action_2}")

    #Step 3:Agent moved Right and finds Room B is dirty.
    percept_3=('B','Dirty')
    action_3=agent.act(percept_3)
    print(f"Percept:{percept_3}->Action:{action_3}")
```

**Output:-**

```
Percept:('A', 'Dirty')->Action:Right
Perecpt:('A', 'Clean')->Action:Right
Percept:('B', 'Dirty')->Action:Right
```

# Practical No.1(Taxi Driver)

**Code:-**

```
class TableDrivenTaxiAgent:
    def __init__(self):
        self.percept_history=[]
        #Percept Format:(Tarffic Signal,Obstacle)
        self.table={
            #Time Step 1: Single percept sequence
            (('Red','No'),):'Stop',
            (('Green','No'),):'Move Forward',
            (('Yellow','No'),):'Slow Down',
            (('Green','Yes'),):'Brake',

            #Time Step 2: Two percept sequence
            (('Red','No'),
             ('Green','No')):'Move Forward',
            (('Green','No'),
             ('Green','Yes')):'Brake',
            (('Green','Yes'),
             ('Green','No')):'Move Forward',

            #Time Step 3: Three percept sequenece
            (('Red','No'),
             ('Green','No'),
             ('Green','Yes')):'Brake',
            (('Green','No'),
             ('Green','Yes'),
             ('Green','No')):'Move Forward'
            }

    def act(self,current_percept):
        #Store current percept
```

```
        self.percept_history.append(current_percept)
        #Convert history to tuple
        history_key=tuple(self.percept_history)

        action=self.table.get(history_key,'Wait')
        return action

if __name__=="__main__":
    agent=TableDrivenTaxiAgent()
    percept_1=('Red','No')
    action_1=agent.act(percept_1)
    print(f"Percept:{percept_1}->Action:{action_1}")

    #Step 2
    percept_2=('Green','No')
    action_2=agent.act(percept_2)
    print(f"Percept:{percept_2}->Action:{action_2}")

    #Step 3
    percept_3=('Green','Yes')
    action_3=agent.act(percept_3)
    print(f"Percept:{percept_3}->Action:{action_3}")
```

**Output:-**

```
Percept:('Red', 'No')->Action:Stop
Percept:('Green', 'No')->Action:Move Forward
Percept:('Green', 'Yes')->Action:Brake
```

## Prroblem Solving by Searching(Uninformed Search)

## Practical No.2(A)

**Aim:-** Given an initial configuration of the 8-puzzle and a goal configuration. Write a python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search(BFS).

**Code:-**

```
from collections import deque

MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]


def get_neighbors(state):

    neighbors = []
    # Find the index of the empty tile(0)
    idx = state.index(0)
    row, col = idx // 3, idx % 3

    for dr, dc in MOVES:
        new_row, new_col = row + dr, col + dc

        # Check if the move stays within the 3x3 grid
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col

            # Create a new state by swapping the empty tile with the target tile
            new_state = list(state)
            new_state[idx], new_state[new_idx] = (
                new_state[new_idx],
                new_state[idx],
            )
            neighbors.append(tuple(new_state))

    return neighbors


def solve_8_puzzle(start, goal):

    # Create lists to tuple because tuples are hashable(can be put in sets)
    start_tuple = tuple(start)
```

```
    goal_tuple = tuple(goal)

    if start_tuple == goal_tuple:
        return [start_tuple]

    # Queue stores tuples of (current_state,path_taken_to_reach_it)
    queue = deque([(start_tuple, [start_tuple])])
    visited = {start_tuple}

    while queue:
        current_state, path = queue.popleft()

        # Check all possible valid moves from the current state
        for neighbor in get_neighbors(current_state):
            if neighbor == goal_tuple:
                return path + [neighbor]

            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None  # Return None if the goal state is unreachable


def print_board(state):

    for i in range(0, 9, 3):
        print(list(state[i:i + 3]))
    print()


# 0 represents the empty space
initial_state = [1, 2, 3, 4, 0, 5, 7, 8, 6]

goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]

print("Solving the puzzle...\n")
solution_path = solve_8_puzzle(initial_state, goal_state)

if solution_path:
    print(f"Success! Shortest path found in {len(solution_path) - 1} moves.\n")
    for step, state in enumerate(solution_path):
        print(f"Step {step}:")
        print_board(state)
```

```
else:
    print("No solution exists for this configuration.")
```

**Output:-**

```
Solving the puzzle...

Success! Shortest path found in 2 moves.

Step 0:
[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

Step 1:
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

# Practical No.2(B)

**Aim:-** Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

**Code:-**

```
J1_CAP = 4
J2_CAP = 3
TARGET = 2

def get_next_states(state):
    j1, j2 = state
    moves = []

    #1.Fill a jug completely
    moves.append((J1_CAP, j2))   #Fill Jug1
    moves.append((j1, J2_CAP))   #Fill Jug2

    #2.Empty a jug completely
    moves.append((0, j2))        #Empty Jug1
    moves.append((j1, 0))        #Empty Jug2

    #3.Pour from Jug1 to Jug2
    pour_to_j2 = min(j1, J2_CAP - j2)
    moves.append((j1 - pour_to_j2, j2 + pour_to_j2))

    #4.Pour from Jug2 to Jug1
    pour_to_j1 = min(j2, J1_CAP - j1)
    moves.append((j1 + pour_to_j1, j2 - pour_to_j1))

    return list(set(moves))

def solve_water_jug_dfs():
    stack = [((0, 0), [(0, 0)])]
    visited = {(0, 0)}

    while stack:
        (j1, j2), path = stack.pop()   #Pop the latest element (LIFO behaviour)

        #Check if either jug has reached the target quantity
        if j1 == TARGET or j2 == TARGET:
            return path
```

```
        for next_state in get_next_states((j1, j2)):
            if next_state not in visited:
                visited.add(next_state)
                stack.append((next_state, path + [next_state]))

    return None

solution = solve_water_jug_dfs()

if solution:
    print(f"Solution found using DFS ({len(solution)-1} steps):")
    for step, (j1, j2) in enumerate(solution):
        print(f"Step {step}: Jug1={j1}L, Jug2={j2}L")
else:
    print(f"It is impossible to get exactly {TARGET}L with these jugs.")
```

**Output:-**

```
Solution found using DFS (4 steps):
Step 0: Jug1=0L, Jug2=0L
Step 1: Jug1=0L, Jug2=3L
Step 2: Jug1=3L, Jug2=0L
Step 3: Jug1=3L, Jug2=3L
Step 4: Jug1=4L, Jug2=2L
```